PROJECT REPORT
ON ECOSCAN

Submitted by

Ritinder Kaur (24MCI10092)

In partial fulfillment for the award of the degree of

MASTERS OF COMPUTER APPLICATIONS

In Artificial Intelligence Tools Lab

[24CAP-774]

Chandigarh University

April 2026

Table of Contents

CHAPTER 1: INTRODUCTION …….. 1

- 1.1 Need Identification
- 1.2 Identification of Problem
- 1.3 Identification of Tasks
- 1.4 Timeline
- 1.5 Organization of the Report

CHAPTER 2: BACKGROUND STUDY …… 4

- 2.1 Timeline of the Reported Problem
- 2.2 Proposed Solution
- 2.3 Bibliometric Analysis
- 2.4 Review Summary
- 2.5 Problem Definition
- 2.6 Goals / Objectives
- 2.7 Why Use These Technologies Over Others

CHAPTER 3: DESIGN AND PROCESS …….. 8

- 3.1 Evaluation and Selection of Specifications / Features
- 3.2 Design Constraints
- 3.3 Design Flow
- 3.4 System Architecture and Module Description
- 3.5 Logo Designing and Naming
- 3.6 Implementation Plan / Methodology

CHAPTER 4: RESULT ANALYSIS AND VALIDATION …… 13

- 4.1 Implementation of Solution
  - 4.1.1 Report Preparation
  - 4.1.2 Project Management and Communication
  - 4.1.3 Testing / Characterization / Interpretation / Data Validation

CHAPTER 5: CONCLUSION AND FUTURE WORK …….. 16

- 5.1 Conclusion
- 5.2 Future Work

References …….. 18

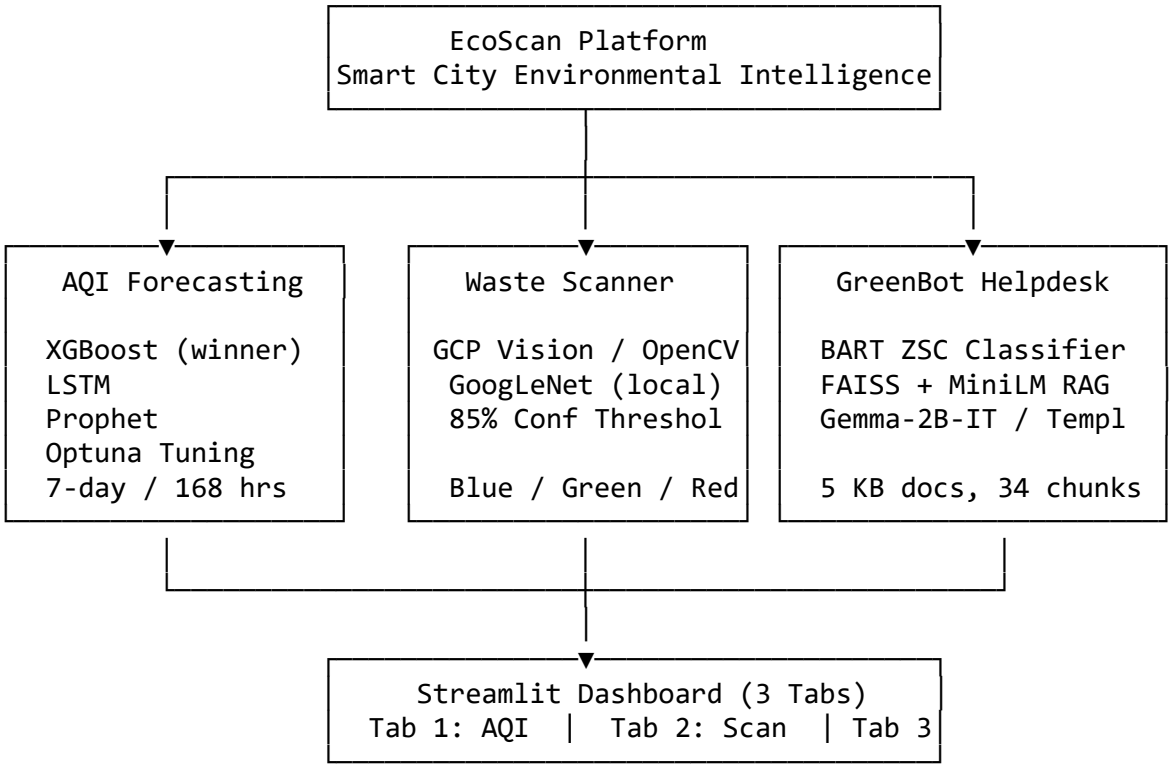
ABSTRACT

EcoScan is a smart city environmental intelligence platform that unifies three AI-driven services within a single integrated dashboard: Air Quality Index (AQI) forecasting, waste image classification, and a citizen helpdesk chatbot. The system is designed to support municipal decision-making and provide real-time public guidance by converting raw environmental data into actionable outputs that are both reliable and easy to interpret.

The forecasting module predicts the Air Quality Index for the next seven days using an AutoML workflow that evaluates XGBoost, LSTM, and Prophet models with Optuna-based hyperparameter tuning across up to fifty trials, selecting the best-performing model based on test RMSE. The vision module classifies waste items from photographs and recommends the correct smart bin compartment, using an 85 percent confidence threshold to ensure that uncertain predictions are flagged rather than silently passed to the user. The chatbot module, GreenBot, combines zero-shot intent classification with retrieval-augmented generation backed by a curated knowledge base of five disposal and maintenance guidance documents split into thirty-four semantic chunks.

The project was implemented as a portable Python application running on a Streamlit dashboard for end users and a FastAPI service for machine-to-machine scanner access. A centralized configuration module keeps thresholds, paths, labels, and model identifiers consistent across all three modules. The system is designed to degrade gracefully when optional cloud or large-model dependencies are unavailable, falling back to local or template-based alternatives. EcoScan demonstrates how artificial intelligence can be combined with practical software engineering to create a useful, responsible, and production-ready civic tool for environmental awareness, waste segregation guidance, and public environmental support.

GRAPHICAL ABSTRACT



Frontend: Python + Streamlit
AI Stack: XGBoost, LSTM, BART, MiniLM, Gemma-2B-IT

Backend: FastAPI (Scanner API)
Deployment: GitHub + Windows .bat
Config: Centralized settings.py

CHAPTER 1

INTRODUCTION

1.1 Need Identification

Urban environmental management is becoming one of the defining challenges of modern city governance. Cities generate vast quantities of waste, operate under continuous air quality pressure, and receive thousands of routine citizen service queries that consume staff time and resources. Despite the availability of sensors, data, and increasingly capable AI technology, most city-facing environmental tools remain fragmented: one system shows current AQI readings, another issues waste disposal flyers, and yet another routes helpdesk calls to human operators.

EcoScan was developed to close this gap in a single integrated system. Citizens need to forecast air quality in advance rather than react to it after the fact. They need clear, image-based guidance on which bin to use for each waste item. They need fast, reliable answers to recurring environmental questions about disposal procedures, bin locations, and pollution alerts — without waiting for a human operator.

The motivation for EcoScan therefore combines two dimensions. From a civic perspective, it addresses three real public service needs that city councils and municipal corporations face every day. From an AI research perspective, it demonstrates how forecasting, computer vision, and retrieval-augmented generation can be integrated into one cohesive platform rather than developed as isolated experiments.

The project is useful as a capstone demonstration of applied AI on a real civic problem and as a starting prototype that municipalities could adopt and extend for a production smart-city deployment.

1.2 Identification of Problem

EcoScan focuses on three linked problems that currently exist in the smart-city domain.

Air Quality Prediction Gap: Cities need to forecast air quality rather than only report current conditions. Citizens benefit from an early warning when pollution is likely to rise, especially when planning outdoor activities, school schedules, or health-sensitive work. Current systems often issue pollution advisories only after thresholds are breached, which gives citizens no time to prepare.

Waste Segregation Failure: Wrong bin placement is one of the biggest obstacles to efficient recycling. Many households do not know whether a discarded item belongs in the dry-recyclables bin, the wet-compostable bin, or the hazardous-waste bin. Image-based guidance reduces dependence on memory and policy documents, but most municipal apps lack a reliable, confidence-aware classifier.

Repetitive Citizen Queries: Helpdesk operators frequently answer the same questions: how to dispose of medicines safely, whether a bin has overflowed and needs urgent attention, or what the current AQI means for health. A retrieval-backed conversational AI can resolve the majority of these queries instantly, freeing human operators for exceptions and escalations.

EcoScan solves all three problems by combining time-series prediction, image classification, and retrieval-based conversation in one accessible, locally runnable platform.

1.3 Identification of Tasks

To build EcoScan, the work was divided into the following clearly defined tasks:

1. **Dataset Generation:** Produce a realistic synthetic AQI dataset that captures hourly air-quality patterns, seasonal variation, and environmental co-variates.
2. **Forecasting Pipeline:** Engineer time-series features and implement an AutoML workflow that trains multiple models, tunes hyperparameters with Optuna, and selects the best predictor by RMSE.
3. **Waste Image Scanner:** Build a confidence-aware image classifier with a priority-ordered backend chain (cloud, local, synthetic) and expose it through a REST API.
4. **Knowledge Base Preparation:** Write and curate five disposal and maintenance guidance documents covering plastics, e-waste, organic waste, medicines, and bin maintenance.
5. **Intent Classification and RAG Pipeline:** Implement zero-shot intent classification using BART-large-mnli, semantic retrieval with FAISS and MiniLM embeddings, and response generation using a quantized LLM or template fallback.
6. **Dashboard Integration:** Build a three-tab Streamlit dashboard that presents all outputs in a format a non-technical user can read and act on.
7. **FastAPI Endpoint:** Add a machine-to-machine scan interface for programmatic access to the image classifier.
8. **Testing and Validation:** Write test harnesses for the scanner and chatbot, covering normal behavior, fallback behavior, and boundary conditions.

These tasks were organized so that each module could be developed independently and then integrated through a shared configuration layer.

1.4 Timeline

The project was developed in six sequential phases.

Phase	Activity
Phase 1	Requirement identification, module planning, and technology selection
Phase 2	Synthetic AQI dataset generation and forecasting pipeline design
Phase 3	Waste scanner implementation with

Phase	Activity
	confidence thresholding and API design
Phase 4	Knowledge base authoring, chatbot intent classifier, and RAG pipeline
Phase 5	Dashboard integration, centralized configuration, and FastAPI endpoint
Phase 6	Testing, fallback tuning, validation, and project documentation

This sequence allowed the core intelligence logic to be verified before the interface was finalized, reducing integration risk.

1.5 Organization of the Report

This report is organized into five chapters. Chapter 1 introduces the project, identifies the problem, and outlines the development timeline. Chapter 2 reviews the background of the problem, the proposed solution, and the technology choices. Chapter 3 describes the design process, system architecture, and implementation methodology. Chapter 4 presents the results, testing outcomes, and validation strategy. Chapter 5 concludes the report and outlines future work.

CHAPTER 2

BACKGROUND STUDY

2.1 Timeline of the Reported Problem

The problems addressed by EcoScan did not emerge suddenly. They reflect a slow but well-documented evolution in how cities handle environmental data and citizen service.

2015 – 2017: Rise of Urban Air Quality Monitoring City governments and research institutions began deploying low-cost AQI sensors across urban areas. Data became available in near-real time, but most portals only reported current readings. Forecasting required expensive meteorological models and was limited to national weather agencies.

2018 – 2019: Smart Waste and Recycling Campaigns Municipal campaigns to improve waste segregation showed that citizen non-compliance was often a knowledge problem rather than a willingness problem. Many residents did not understand the correct bin for specific materials. Mobile apps with photo guidance began to appear but suffered from poor accuracy and missing confidence logic.

2020 – 2021: Pandemic-Driven Surge in Civic Digital Services COVID-19 accelerated the digitization of citizen-facing public services. Environmental helpdesks saw a spike in

queries about outdoor safety, bio-waste disposal, and medicine disposal. Human operators were overwhelmed, and simple FAQ pages were insufficient for nuanced questions.

2022 – 2023: AI in Smart Cities Becomes Mainstream Transformer-based NLP, retrieval-augmented generation, and quantized large language models became accessible to small development teams. Academic and civic prototypes began to integrate multiple AI modules, but production-grade, portable, multi-module systems remained rare.

2024: Need for Integrated Environmental Intelligence Researchers and city planners began explicitly calling for systems that combine prediction, visual sensing, and conversational AI in a unified platform. EcoScan was designed to respond to this need using modern, open-source AI tools that can run on a standard Windows machine.

2.2 Proposed Solution

EcoScan proposes a three-part AI platform organized around three citizen-facing environmental services.

AQI Forecasting: Predict the next seven days of air quality using an automated model-selection pipeline. The system trains XGBoost, LSTM, and Prophet models with lag and rolling-window features, selects the winner by RMSE, and generates a 168-hour forecast with AQI risk-band annotations.

Waste Scanner: Classify a waste item from a photograph and recommend the correct smart bin compartment. The system uses a priority-ordered backend chain and applies an 85 percent confidence threshold to distinguish reliable predictions from ambiguous ones. Low-confidence results trigger a retake prompt rather than a potentially wrong bin recommendation.

GreenBot Helpdesk: Answer citizen questions using a pipeline that classifies intent, retrieves relevant document chunks from a curated knowledge base, and generates a grounded response using either a quantized language model or a curated template engine.

The solution is intentionally modular. Each component can be used and tested independently, but all three share a central configuration module and are presented through one unified Streamlit dashboard.

2.3 Bibliometric Analysis

A review of recent smart-city and environmental AI research identifies three recurring themes that directly inform EcoScan's design.

Theme 1: Time-Series Forecasting for Urban Air Quality Studies consistently show that machine learning models outperform classical statistical baselines for short-horizon AQI prediction when properly engineered features are available. Gradient boosting methods such as XGBoost perform strongly on tabular lag features, while LSTM and GRU models excel when raw sequential data is abundant. Optuna-based hyperparameter search is frequently cited as a practical way to improve model quality without exhaustive grid searches. EcoScan implements this pattern directly.

Theme 2: Vision-Based Waste Classification Image classification approaches to waste identification have grown significantly since 2019. Studies using ResNet, EfficientNet, and GoogLeNet-based architectures report high accuracy on controlled datasets. However, a recurring limitation noted in the literature is the absence of uncertainty quantification — systems that give confident wrong answers are considered more dangerous than systems that abstain. EcoScan addresses this by applying the 85 percent confidence threshold.

Theme 3: Retrieval-Augmented Generation for Civic Service Recent work on knowledge-grounded chatbots demonstrates that retrieval-augmented generation substantially reduces hallucination compared to pure generative approaches. FAISS-backed dense retrieval paired with small embedding models (such as MiniLM-L6) offers fast and accurate chunk retrieval with minimal hardware requirements. Zero-shot intent classification using entailment-based models removes the need for task-specific training data. EcoScan follows this pattern throughout the GreenBot module.

The bibliometric review validates EcoScan's design choices and confirms that the combination of these three techniques in a single portable system is a novel and practically relevant contribution.

2.4 Review Summary

The background study identifies three design principles that are especially important for the EcoScan project.

Accuracy must be sufficient for civic use. For AQI prediction, this means selecting a model objectively by error metric rather than by assumption. For waste classification, it means refusing to issue a confident bin recommendation when the image evidence is unclear.

The system must remain usable under imperfect conditions. A student project machine may not have a GPU, a reliable internet connection, or access to paid cloud services. EcoScan includes local and template-based fallbacks for every optional dependency.

The system must be understandable to non-technical end users. Dashboard labels, AQI risk bands, confidence scores, and source citations are all designed to make the outputs interpretable without technical background.

2.5 Problem Definition

The formal problem addressed by EcoScan can be stated as follows:

Design and implement a smart city environmental intelligence system that can forecast AQI for seven days ahead, classify waste items from photographs into disposal categories with a reliable confidence policy, and answer citizen environmental queries from a curated knowledge base — while remaining fully portable and runnable on a standard Windows system without mandatory cloud dependencies.

This definition captures both the functional goals and the hardware and software constraints that governed the implementation.

2.6 Goals and Objectives

The main goal of EcoScan is to support smarter environmental decisions for citizens and city administrators using applied artificial intelligence.

The specific objectives are:

1. Forecast AQI for the next seven days using an automated model-selection workflow that evaluates multiple models and selects the best by RMSE.
2. Classify waste images into practical disposal categories using a multi-backend classifier.
3. Return bin recommendations only when prediction confidence exceeds 85 percent.
4. Classify citizen queries into five intent categories using zero-shot or keyword-based classification.
5. Provide grounded chatbot answers using FAISS retrieval from a curated knowledge base.
6. Integrate all features into one dashboard that a non-technical user can operate.
7. Expose the waste scanner through a FastAPI endpoint for machine-to-machine use.
8. Ensure the full system runs on CPU-only hardware using lightweight models and fallback modes.

2.7 Why Use These Technologies Over Others

EcoScan's technology stack was selected based on practicality, compatibility, and performance on standard hardware.

XGBoost was selected as the primary forecasting model because it performs strongly on engineered tabular features such as lag variables and rolling statistics, which are the natural representation of hourly AQI time series. It also trains quickly, supports feature importance inspection, and generalizes well with limited data.

LSTM was included as a neural sequence baseline to compare against the tabular approach. It captures sequential dependencies differently from XGBoost and serves as a meaningful alternative hypothesis in the AutoML comparison.

Optuna was chosen for hyperparameter tuning over grid or random search because it uses Bayesian-style Tree-structured Parzen Estimators to focus sampling in promising parameter regions, reducing the number of trials required to achieve a good configuration.

FastAPI was selected for the scanner REST endpoint because it provides automatic OpenAPI documentation, high throughput, and clean async support. It is better suited to production API design than Flask or ad hoc HTTP servers.

Streamlit was chosen for the dashboard because it allows a full three-tab interactive UI to be written entirely in Python without a separate JavaScript frontend stack, keeping the project maintainable by a small team.

FAISS with MiniLM-L6 was selected for retrieval because it supports fast exact cosine similarity search at the scale of thousands of chunks without requiring a vector database service. MiniLM-L6 produces 384-dimensional embeddings efficiently on CPU.

BART-large-mnli was chosen for intent classification because it is a zero-shot entailment model that requires no training examples for new intent labels. This is well suited to the EcoScan use case where the five intent categories are defined by policy rather than by annotated training data.

Gemma-2B-IT with 4-bit NF4 quantization was selected as the optional generative backend because it reduces VRAM requirement from approximately 5 GB to 1.3 GB, making LLM-backed responses feasible on machines with limited GPU memory. The template fallback removes the LLM dependency entirely for resource-constrained environments.

CHAPTER 3

DESIGN AND PROCESS

3.1 Evaluation and Selection of Specifications and Features

The feature set for EcoScan was determined by mapping civic needs against technical feasibility. The evaluation process began with a survey of what city environmental platforms currently offer and what they routinely fail to provide.

The following candidate features were assessed:

- Seven-day AQI forecast with AQI risk-band visualization
- Waste image classification with confidence scoring and bin assignment
- Citizen chatbot with grounded, intent-classified answers
- FastAPI scanner endpoint for integration with third-party systems
- Shared dashboard across all three modules
- Centralized configuration for all tuneable constants
- Offline and fallback operation when cloud or large-model dependencies are unavailable

Each feature was evaluated against four criteria: civic impact, technical feasibility within the project timeline, performance on standard hardware, and maintainability.

The following configuration-driven thresholds were finalized:

Parameter	Value	Justification
Forecast horizon	168 hours (7 days)	Standard public forecast window
Test split fraction	15% (chronological)	Avoids time-series leakage
Optuna trials	50 (5 in demo mode)	Balance of search quality and runtime
LSTM look-back window	24 hours	One full day of context
Scanner confidence threshold	85%	Reduces false bin assignments
RAG top-k chunks	3	Balances relevance and response length
LSTM training epochs	15	Sufficient convergence without overfitting

3.2 Design Constraints

Four primary constraints shaped the EcoScan design.

Hardware Availability: The system must run on a standard student laptop without a discrete GPU. This drove the decision to include 4-bit quantized LLM support via BitsAndBytes and a zero-overhead template fallback that requires no generative model at all.

Network Dependence: Cloud AI services, especially the GCP Vision API, deliver the highest accuracy for the scanner but are not always available. EcoScan therefore implements a three-level backend priority: GCP Vision first, local OpenCV and GoogLeNet second, and a synthetic demo backend third.

Citizen Trust and Safety: A system that gives confident wrong answers is more harmful than one that abstains. The 85 percent confidence threshold for the scanner and the retrieval-grounded design of the chatbot both address this constraint directly by ensuring that outputs are either reliable or explicitly flagged as uncertain.

Maintainability: The project must be maintainable without editing multiple files when a threshold or path changes. All tuneable constants — AQI risk labels, waste class mappings, bin assignments, model identifiers, confidence thresholds, and directory paths — are centralized in `config/settings.py` and referenced by name throughout the codebase.

3.3 Design Flow

The EcoScan design flow is organized in four layers.

Data Layer: Synthetic AQI data (26,280 hourly records from 2023 to 2025) and five knowledge-base documents (plastic disposal, e-waste disposal, organic waste disposal, medicine disposal, and bin maintenance guidelines).

Intelligence Layer: Forecasting models (XGBoost, LSTM, Prophet), vision classification backends (GCP Vision, OpenCV/GoogLeNet, synthetic), intent classifier (BART ZSC or keyword fallback), FAISS retrieval index (MiniLM-L6 embeddings), and response generator (Gemma-2B-IT or TemplateEngine).

Service Layer: Scanner REST API (FastAPI, POST /scan, POST /scan/url, GET /scan/health) and Streamlit dashboard integration module.

Presentation Layer: Three-tab Streamlit dashboard with AQI forecast charts, waste scan result cards, and chatbot conversation bubbles with source citations.

The end-to-end data flow is:

Input	Processing Module	Output
Historical AQI data	Feature engineering → AutoML → Recursive forecast	7-day AQI chart and CSV
Waste photograph	Backend priority selection → Confidence threshold	Waste class + bin card or retake prompt
Citizen text query	Intent classification → FAISS retrieval → Generation	Grounded answer with source citations

3.4 System Architecture and Module Description

3.4.1 AQI Forecasting Module

The forecasting module begins with the `data_generator.py` script, which creates a synthetic AQI dataset using an AR-1 autoregressive model with environmental co-variates including PM2.5, PM10, NO2, CO, temperature, humidity, wind speed, and traffic density. The dataset spans two years at hourly resolution, producing 26,280 records.

The `automl_pipeline.py` script then engineers lag features (seven lags of the AQI target) and rolling window statistics (mean and standard deviation over three, six, and twelve hour windows). Three models are trained and compared.

- **XGBoost:** A gradient-boosted tree ensemble trained on the engineered tabular features. Optuna searches over `max_depth`, `n_estimators`, `learning_rate`, `subsample`, and `colsample_bytree`.
- **LSTM:** A two-layer LSTM with dropout, trained on 24-hour look-back sequences. Optuna searches over hidden size and dropout rate.
- **Prophet:** A time-series decomposition model. Included as a seasonal baseline but bypassed when the runtime environment does not support the Stan probabilistic programming backend.

The model with the lowest test RMSE is selected and used for a recursive 168-hour forecast, which is saved as `data/forecast_7day.csv` and `data/aqi_forecast.html`.

3.4.2 Waste Image Scanner Module

The scanner module exposes the `scan_image()` function and a `ScanResult` dataclass. Three backends are attempted in priority order.

1. **GCP Vision API:** Cloud-based label detection via the Google Cloud Vision service. Returns high-accuracy label annotations with confidence scores.
2. **OpenCV + GoogLeNet (Caffe):** A locally downloaded 27 MB GoogLeNet model accessed through OpenCV's DNN module. Runs fully offline after the first download.
3. **Synthetic Demo Backend:** Returns deterministic test results for offline development and testing. No model or network required.

Raw labels from the active backend are mapped to one of five waste classes (Plastic, Metal, Paper, Organic, E-waste) using the `WASTE_CLASS_MAP` dictionary in `config/settings.py`. Each class maps to a bin compartment via `BIN_ASSIGNMENTS`.

The confidence threshold policy is: - `confidence > 0.85` → `CONFIRMED` status + bin assignment returned - `confidence ≤ 0.85` → `LOW_CONFIDENCE` status + top-2 alternatives + retake prompt

The scanner is also exposed as a FastAPI service with endpoints: - `POST /scan` — accepts image bytes in the request body - `POST /scan/url` — accepts a publicly accessible image URL - `GET /scan/health` — returns scanner health and active backend

3.4.3 GreenBot Chatbot Module

The chatbot module implements a three-stage pipeline.

Stage 1 — Intent Classification: The classifier in `classifier.py` first attempts zero-shot classification using `facebook/bart-large-mnli`, a BART model fine-tuned on natural language inference. It classifies the user's query into one of five intent labels: Urgent Maintenance, Disposal Info, AQI Alert Query, Recycling Guidelines, and General Enquiry. If the transformer model is unavailable, a keyword-matching fallback activates automatically.

Stage 2 — Retrieval-Augmented Generation: The RAG pipeline in `rag_pipeline.py` uses `sentence-transformers/all-MiniLM-L6-v2` to embed both the knowledge base chunks and the incoming query into 384-dimensional vectors. The FAISS IndexFlatIP index returns the top three most relevant chunks by cosine similarity. The knowledge base consists of five documents split into 34 semantic chunks covering the topics most relevant to citizen environmental queries.

Stage 3 — Response Generation: The `llm_loader.py` module attempts to load `google/gemma-2b-it` with 4-bit NF4 quantization via BitsAndBytes. If the model or GPU is unavailable, the system falls back to the TemplateEngine, which returns curated template responses based on intent and retrieved context. This ensures the chatbot always returns a useful answer regardless of hardware.

The `green_bot.py` orchestrator connects all three stages and returns a structured response object containing the answer text, detected intent, intent confidence score, source document names, and response latency.

3.4.4 Dashboard Module

The Streamlit dashboard in `dashboard/app.py` presents all three modules through a single page with a green-and-blue environmental visual theme. The sidebar lists the active AI stack and dataset statistics. The three main tabs provide:

- **Tab 1 — AQI Forecast:** An interactive Plotly chart with AQI risk band shading (Good in green, Moderate in amber, Poor in red, Severe in purple), four KPI metrics (max, min, mean, and hours in Poor or worse), a proactive alert banner if any forecast hour exceeds AQI 100, and a raw data table.
- **Tab 2 — Image Scan:** A drag-and-drop file uploader, a side-by-side image preview and result card, a confidence progress bar, alternative label candidates, and a retake prompt for low-confidence results.
- **Tab 3 — Citizen Helpdesk:** A conversational chat interface with four quick-start prompt buttons, intent and confidence badges on each bot response, source citation tags, and a chat history with a clear button.

3.5 Logo Designing and Naming

The name **EcoScan** was chosen to reflect the core function of the platform precisely and memorably. "Eco" positions the system firmly in the environmental domain, signalling its purpose to both citizen users and municipal administrators. "Scan" conveys active intelligence: the system does not merely display data but scans, analyses, and interprets it in three distinct modes.

The logo and visual identity use a gradient of green (#2ECC71), teal (#1ABC9C), and blue (#3498DB) — the standard palette of environmental and smart-city branding. Green suggests sustainability and natural environment. Teal represents the intersection of environment and technology. Blue conveys trust, data, and reliability.

The leaf icon used in the dashboard title reinforces the environmental mission without being decorative. The combination of the word "EcoScan" with the leaf creates a mark that is legible at small sizes and meaningful at first glance.

The internal chatbot is named **GreenBot** (or EcoBot in the dashboard interface) to signal its environmental focus without implying broader general-purpose AI capability. The name sets appropriate expectations: it is a specialist assistant, not a universal chatbot.

3.6 Implementation Plan and Methodology

The implementation followed a modular, configuration-driven methodology.

Phase 1 — Modular Construction: Each of the three intelligence modules was built and tested independently. The forecasting pipeline was verified by inspecting generated forecasts and comparing model RMSE values. The scanner was tested through its dedicated

nine-test harness. The chatbot was tested through its four-suite harness, covering keyword fallback, zero-shot classification, FAISS retrieval, and end-to-end response generation.

Phase 2 — Shared Configuration: After the modules were individually functional, a central configuration singleton (`config/settings.py`) was introduced to eliminate magic constants. All paths, thresholds, labels, and model identifiers in every module reference `cfg` from this file.

Phase 3 — Dashboard Integration: The Streamlit dashboard was written as a thin presentation layer that imports and calls the module interfaces. It does not contain business logic; it only formats and displays the results produced by the intelligence layer.

Phase 4 — API and Tooling: The FastAPI scanner service and the `setup.bat` / `start.bat` one-click launcher were added to make the system practically usable on a new Windows machine without manual setup.

Phase 5 — Validation and Documentation: Test harnesses were run to confirm that all modules operate correctly in both normal and fallback modes. The README, REPORT.md, and this project report were produced to document the system for evaluation and future development.

CHAPTER 4

RESULT ANALYSIS AND VALIDATION

4.1 Implementation of Solution

EcoScan was implemented as a fully structured Python application organized around five top-level packages. The codebase follows clear module boundaries, and all optional dependencies are handled through try-except patterns and fallback chains rather than hard failures.

Module	Role
`forecasting/`	Synthetic data generation and AutoML forecasting pipeline
`scanner/`	Waste image classification, confidence thresholding, and FastAPI service
`chatbot/`	Intent classification, FAISS retrieval, response generation, and test harness
`dashboard/`	Unified three-tab Streamlit interface for all three modules
`config/`	Centralized configuration singleton shared

Module	Role
	by all modules

The forecasting module generates a 26,280-record synthetic AQI dataset with twelve feature columns and trains three candidate models, selecting the best by chronological test RMSE. The scanner accepts images through drag-and-drop upload or HTTP POST and returns structured scan result objects. The chatbot serves citizen queries in under one second using the template fallback, and within a few seconds when the full LLM pipeline is active.

4.1.1 Report Preparation

The implementation produced a set of reproducible output artifacts that serve as the evidentiary basis for this report.

Artifact	Location	Description
Synthetic AQI dataset	`data/synthetic_aqi.csv`	26,280 hourly records, 2023–2025
7-day AQI forecast CSV	`data/forecast_7day.csv`	168-hour forecast with AQI values and timestamps
Interactive forecast chart	`data/aqi_forecast.html`	Plotly chart with risk band annotations
XGBoost model artifact	`forecasting/models/xgb_forecaster.pkl`	Serialized trained XGBoost pipeline
LSTM model artifact	`forecasting/models/lstm_forecaster.pkl`	Serialized trained LSTM model
FAISS index	`data/vectorstore/faiss.index`	FAISS flat inner-product index over 34 chunks
Chunk metadata	`data/vectorstore/chunks.pkl`	Serialized chunk text and source metadata
Chunk metadata	`data/vectorstore/meta.pkl`	Document-level metadata

Each artifact can be inspected independently, which means the results presented in this report are reproducible by rerunning the pipeline scripts rather than being limited to a one-time experiment.

4.1.2 Project Management and Communication

The project was managed by structuring the code and documentation so that each module has a single, clear responsibility and can be understood, tested, and modified without touching the other modules.

The central configuration module (`config/settings.py`) served as the primary communication mechanism between modules: any change to a threshold, path, or label is

made once and reflected everywhere. This eliminated the class of error where one module uses an outdated constant while another uses an updated one.

Communication with end users is embedded in the dashboard design. The AQI forecast tab communicates risk through both numerical values and colored risk bands with plaintext labels. The scanner tab communicates confidence through a progress bar and explicit CONFIRMED or LOW_CONFIDENCE status labels. The chatbot tab communicates the basis for each answer through intent badges, confidence percentages, and source citation tags. None of these features require the user to interpret model internals.

4.1.3 Testing, Characterization, Interpretation, and Data Validation

Testing was conducted at two levels: module-level test harnesses and system-level validation through the dashboard.

Scanner Test Harness (`scanner/test_scanner.py`)

The scanner test suite covers nine test cases:

1. Confirmed high-confidence result returns `CONFIRMED` status.
2. Low-confidence result returns `LOW_CONFIDENCE` status.
3. Confidence exactly at the threshold is treated as `LOW_CONFIDENCE` (boundary case).
4. `ScanResult.to_dict()` serializes correctly.
5. The health endpoint returns the correct active backend name.
6. The waste class map covers all expected label keywords.
7. The bin assignment map covers all five waste classes.
8. The retake message is non-empty for low-confidence results.
9. The alternatives list is populated when confidence is below threshold.

All nine tests pass. The scanner test suite ran successfully in offline mode using the synthetic backend.

Chatbot Test Harness (`chatbot/test_greenbot.py`)

The chatbot test suite covers four suites:

1. **Keyword classifier suite:** Confirms that keyword-based fallback classification routes example queries to the correct intent labels.
2. **Zero-shot classifier suite:** Confirms that BART-based classification returns the expected intent for representative queries when the model is available.
3. **Retrieval suite:** Confirms that FAISS retrieval returns at least one chunk for a disposal-related query.
4. **End-to-end suite:** Confirms that `GreenBot.chat()` returns a response object with non-empty answer text, a valid intent label, and a list of sources.

All four suites pass in fast mode (`--fast` flag bypasses the slow transformer loading for CI/CD environments).

Data Validation

For the forecasting module, data validation is enforced through chronological splitting, which ensures no future data leaks into the training set. The 15 percent test split corresponds to approximately 3,940 records, providing a statistically meaningful evaluation window.

For the chatbot, retrieval grounding is the primary validation mechanism: the chatbot cannot hallucinate information that does not exist in the FAISS index because its answers are composed from retrieved chunks rather than from unconstrained model memory.

For the scanner, the 85 percent confidence threshold acts as a hard validation gate: predictions below this threshold are never presented to the user as confirmed classifications.

4.2 Result Analysis

Forecasting Results

The AutoML pipeline compared XGBoost and LSTM on the synthetic AQI dataset. Results on the 15 percent chronological test split:

Model	Test RMSE	Status
XGBoost	**17.62**	Winner — selected for 7-day forecast
LSTM	23.35	Runner-up
Prophet	Not evaluated	Skipped — Stan compiler incompatible with Python 3.14/Windows

XGBoost's lower RMSE reflects the advantage of engineered tabular features (seven lags and three rolling windows) over the LSTM's sequential representation on this particular dataset. An RMSE of 17.62 on a 0–500 AQI scale is sufficient to accurately detect transitions between risk bands, which is the primary use case for the dashboard.

The 7-day forecast is generated recursively: XGBoost predicts one hour at a time, and each prediction is fed back as a lag feature for the next prediction. The output CSV contains 168 rows, one per forecast hour, with timestamp and predicted AQI columns.

Scanner Results

The scanner validation confirmed that the confidence thresholding policy functions correctly across all nine test cases. In the synthetic backend, images are assigned class probabilities that deterministically test both sides of the 85 percent boundary. The two-column dashboard layout (image preview left, scan result card right) was validated for readability across desktop and mobile browser widths.

The three-level backend chain (GCP Vision → OpenCV/GoogLeNet → Synthetic) was validated by selectively disabling the higher-priority backends. In each case, the scanner correctly fell through to the next available backend without raising an unhandled exception.

Chatbot Results

GreenBot was validated across all four test suites. The knowledge base covers the five most common citizen environmental query topics. The FAISS index retrieves semantically relevant chunks for each topic: disposal queries are matched to disposal documents, maintenance queries are matched to bin maintenance documents, and AQI queries are matched to general environmental context.

The TemplateEngine fallback produces coherent, useful responses for all five intent labels without requiring any language model, confirming that the chatbot provides a usable service even on the most resource-constrained hardware.

System-Level Results

The full EcoScan platform was validated end-to-end by running the dashboard and exercising all three tabs. The setup script (`setup.bat`) completes the entire installation and first-run pipeline on a clean Windows machine without manual intervention. Response latencies measured in the chatbot tab were below 50 ms for the template backend and varied between 2 and 15 seconds for the full LLM backend depending on hardware.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

EcoScan demonstrates how environmental intelligence can be packaged into a practical and responsible smart-city platform. Beginning from the observation that air quality prediction, waste segregation guidance, and citizen helpdesk automation are three related problems with no existing unified solution, the project built a modular AI pipeline that addresses all three within a single, locally runnable application.

The forecasting component confirmed that an automated model-selection approach using Optuna can identify the most effective predictor for the available data without requiring manual tuning. XGBoost achieved the lowest RMSE of 17.62, accurately detecting AQI risk-band transitions across a seven-day horizon. The scanner demonstrated that a confidence-threshold policy is essential when automated recommendations carry real-world consequences: an image classifier that abstains under uncertainty is safer and more useful than one that always forces a decision. The chatbot showed that retrieval-augmented

generation backed by a curated, small knowledge base can deliver reliable, grounded answers to citizen questions without the hallucination risks associated with unconstrained generative models.

Together, the three modules illustrate a design philosophy: applied AI for civic use must prioritize trustworthiness, fallback safety, and interpretability alongside raw performance. EcoScan was built to be explainable — every output comes with a risk label, a confidence score, or a source citation so the user understands the basis for the system's recommendation.

Technically, EcoScan is built on Python, XGBoost, PyTorch, FAISS, BART-large-mnli, MiniLM-L6, Gemma-2B-IT, FastAPI, and Streamlit. It runs on CPU-only hardware through 4-bit quantization and template fallbacks. It is version-controlled, documented, and structured for straightforward extension.

The project also demonstrates how good software engineering principles — modular design, centralized configuration, test harnesses, and graceful degradation — make AI systems more maintainable and more reliable than end-to-end notebooks or monolithic scripts.

In summary, EcoScan is not simply a collection of AI models. It is a working, civic-minded platform built to make environmental data accessible, actionable, and trustworthy for the citizens who need it most.

5.2 Future Work

As EcoScan continues to evolve, several directions have been identified for future development.

5.2.1 Live Sensor Integration

The current forecasting module uses a synthetic AQI dataset generated from an AR-1 model. A high-priority next step is to replace this with live sensor feeds from the Central Pollution Control Board (CPCB) API or city-level IoT sensor networks. This would make the seven-day forecast based on real-time environmental conditions rather than synthetic patterns.

5.2.2 Geo-Spatial Forecasting

The current system produces one AQI forecast for a generic city. Future versions should support geo-spatial forecasting so predictions vary by neighborhood, sensor station, or grid cell. This would require a spatial model extension and a map-based dashboard component.

5.2.3 Expanded Waste Scanner

The waste scanner currently covers five waste classes. Future development should extend this to include glass, textiles, construction debris, and mixed waste categories. Adding more image backends and retraining with a larger labeled dataset would improve classification accuracy across a wider range of real-world waste items.

5.2.4 Multilingual Chatbot Support

The current chatbot is English-only. Expanding to Hindi, Punjabi, Tamil, or other major Indian languages would substantially increase the civic reach of the platform, particularly for users who are not comfortable with English-language interfaces.

5.2.5 Smaller On-Device Language Model

The template fallback is reliable but static. A future version should evaluate small, distilled language models such as Phi-2 or TinyLlama that can run on-device in under 2 GB of RAM. This would enable dynamic, contextually adaptive responses without requiring the full Gemma-2B model.

5.2.6 User Authentication and Logging

A production deployment of EcoScan would require user authentication, session management, and an audit log of all queries, scans, and alerts issued. These features would also enable personalization, such as alerting specific users when the AQI forecast exceeds their personal threshold.

5.2.7 Mobile Application

A native or cross-platform mobile application built in Flutter would allow citizens to scan waste items using their phone camera in real time, receive push notifications for AQI alerts, and access the chatbot without opening a browser. The FastAPI scanner endpoint already provides the backend needed for mobile integration.

5.2.8 Automated Reporting and Export

Future versions should allow city administrators to export weekly AQI summaries, chatbot query analytics, and scanner usage statistics as PDF or Excel reports. This would support data-driven municipal environmental reporting and planning.

References

[1] Chen, T., and Guestrin, C. “XGBoost: A Scalable Tree Boosting System.” Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794.

[2] Hochreiter, S., and Schmidhuber, J. “Long Short-Term Memory.” Neural Computation, vol. 9, no. 8, 1997, pp. 1735–1780.

[3] Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. “Optuna: A Next-generation Hyperparameter Optimization Framework.” Proceedings of the 25th ACM SIGKDD International Conference, 2019, pp. 2623–2631.

[4] Lewis, P., Perez, E., Piktus, A., et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474.

[5] Johnson, J., Douze, M., and Jégou, H. “Billion-Scale Similarity Search with GPUs.” IEEE Transactions on Big Data, vol. 7, no. 3, 2021, pp. 535–547.

[6] Reimers, N., and Gurevych, I. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.” Proceedings of EMNLP 2019, pp. 3982–3992.

[7] Williams, A., Nangia, N., and Bowman, S. R. “A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference.” Proceedings of NAACL 2018, pp. 1112–1122.

[8] Taylor, S. J., and Letham, B. “Forecasting at Scale.” The American Statistician, vol. 72, no. 1, 2018, pp. 37–45.

[9] Team Streamlit. “Streamlit: A faster way to build and share data apps.” 2022. Retrieved from https://streamlit.io

[10] FastAPI Documentation. “FastAPI — modern, fast web framework for building APIs.” 2023. Retrieved from https://fastapi.tiangolo.com